

Tabela de Instruções e Diretivas MIPS64

Nuno Veiga, Rui Vasco Monteiro

Instrução	Exemplo	Significado
Add	dadd R1, R2, R3	$R1 = R2 + R3$
Add Immediate	daddi R1, R2, #100	$R1 = R2 + 100$
Subtract	dsub R1, R2, R3	$R1 = R2 - R3$
And Logical Immediate	andi R1, R2, #20	$R1 = R2 \text{ AND } 20$
Or Logical	or R1, R2, R3	$R1 = R2 \text{ OR } R3$
Load (ld, lw, lb) d=64 bits w=32 bits b=8 bits	ld R1, A(R0)	$R1 = A$
	ld R1, A(R2)	$R1 = \text{Memória} [\text{Endereço de } A + R2]$
	ld R1, (R2)	$R1 = \text{Memória} [0+R2]$
	ld R1, 200(R2)	$R1 = \text{Memória} [200+R2]$
Store (sd,sw,sb)	sd R1, 200(R2)	$\text{Memória} [200+R2] = R1$
Doubleword Shift Left Logical	dsll R1, R2, #3	$R1 = R2 \ll 2$ ($R1 = R2 * 2^3$)
Doubleword Shift Right Logical	dsrl R1, R2, #4	$R1 = R2 \gg 4$ ($R1 = R2 / 2^4$)
Doubleword Shift Left Logical Variable	dsllv R1, R2, R3	$R1 = R2 \ll R3$ ($R1 = R2 * 2^{R3}$)
Set Less Than	slt R1, R2, R3	Se $R2 < R3$ faça $R1 = 1$ senão $R1 = 0$
Set Less Than Immediate	slti R1, R2, #20	Se $R2 < 20$ faça $R1 = 1$ senão $R1 = 0$
Load Floating-point (64 bits)	ld F1, 100(R1)	$F1 = \text{memória} [R1 + 100]$
Store Floating-point (64 bits)	sd F2, 100(R1)	$\text{Memória} [R1 + 100] = F2$ (64 bits)
Add Double Precision	add.d F1, F2, F3	$F1 = F2 + F3$ (precisão dupla, 64 bits)
Subtract Single Precision	sub.s F1, F2, F3	$F1 = F2 - F3$ (precisão simples, 32 bits)
Multiply Double Precision	mul.d F1, F2, F3	$F1 = F2 * F3$ (precisão dupla)
Divide Single Precision	div.s F1, F2, F3	$F1 = F2 / F3$ (precisão simples)
Move Floating-point	mov.d F1, F2	$F1 = F2$ (precisão dupla)
Halt	halt	Para a execução do programa
No Operation	nop	Não faz nada
Saltos condicionais		Salta para etiqueta se ...
Branch Equal Zero	beqz R1, etiqueta	... se $R1 = 0$ (mesmo que beq R1,R0, etiqueta)
Branch on Not Equal Zero	bnez R1, etiqueta	... se $R1 \neq 0$ (mesmo que bne R1,R0, etiqueta)
Branch Equal	beq R1, R2, etiqueta	... se $R1 = R2$
Branch on Not Equal	bne R1, R2, etiqueta	... se $R1 \neq R2$
Saltos incondicionais		Salta para ...
Jump	j etiqueta	... etiqueta
Jump Register	jr R31	... a instrução no endereço guardado em R31
Jump and Link	jal etiqueta	... etiqueta e $R31 = PC + 4$ (chamada de procedimento)

Outras instruções:

daddu - add integers unsigned	daddui - add immediate unsigned
lbu - load byte unsigned	dsubu - subtract integers unsigned
lh - load 16-bit half-word	sltu - set if less than unsigned
lhu - load 16-bit half word unsigned	sltiu - set if less than or equal imm. unsigned
sh - store 16-bit half-word	dsra - shift right arithmetic
lwu - load 32-bit word unsigned	dsrlv - shift right logical by variable
lui - load upper half of register immediate	dsrav - shift right arithmetic by variable
movz - move if register equals zero	and - logical and
movn - move if register not equal to zero	xor - logical xor
cvt.w.d - convert 32-bit integer to floating-point	ori - logical or immediate
cvt.l.d - convert 64-bit integer to floating-point	xori - exclusive or immediate
mtc1 - move 32 bit integer from integer register to floating-point register	xori - logical xor immediate

Diretivas para o assembler:

.data	- início do segmento de dados
.text	- início do segmento de programa
.code	- o mesmo que .text
.org <n>	- endereço inicial
.space <n>	- reserva um espaço de n bytes na memória
.asciiz <s>	- define uma cadeia de caracteres terminada por nulo.
.ascii <s>	- define uma cadeia de caracteres
.align <n>	- alinha o próximo endereço para uma fronteira de n-bytes
.word <n1>,<n2>..	- define palavras de 64 bits de dados com valor inicial
.byte <n1>,<n2>..	- define uma sequência de bytes com valor inicial.
.word32 <n1>,<n2>..	- define palavras de 32 bits com valor inicial.
.word16 <n1>,<n2>..	- define palavras de 16 bits com valor inicial.
.double <f1>,<f2>..	- define palavras no formato ponto-flutuante de 64 bits.

Onde <n> denota um número inteiro, <s> significa uma cadeia de caracteres entre aspas e <n1>, <n2> são valores inteiros separados por vírgula e <f1>, <f2> são valores reais separados por vírgula.